



UNGDOMMENS NATURVIDENSKABELIGE FORENING

Aktiveringsfunktioner, Regularisering og Data

Den lille krølning der gør alt forskellen

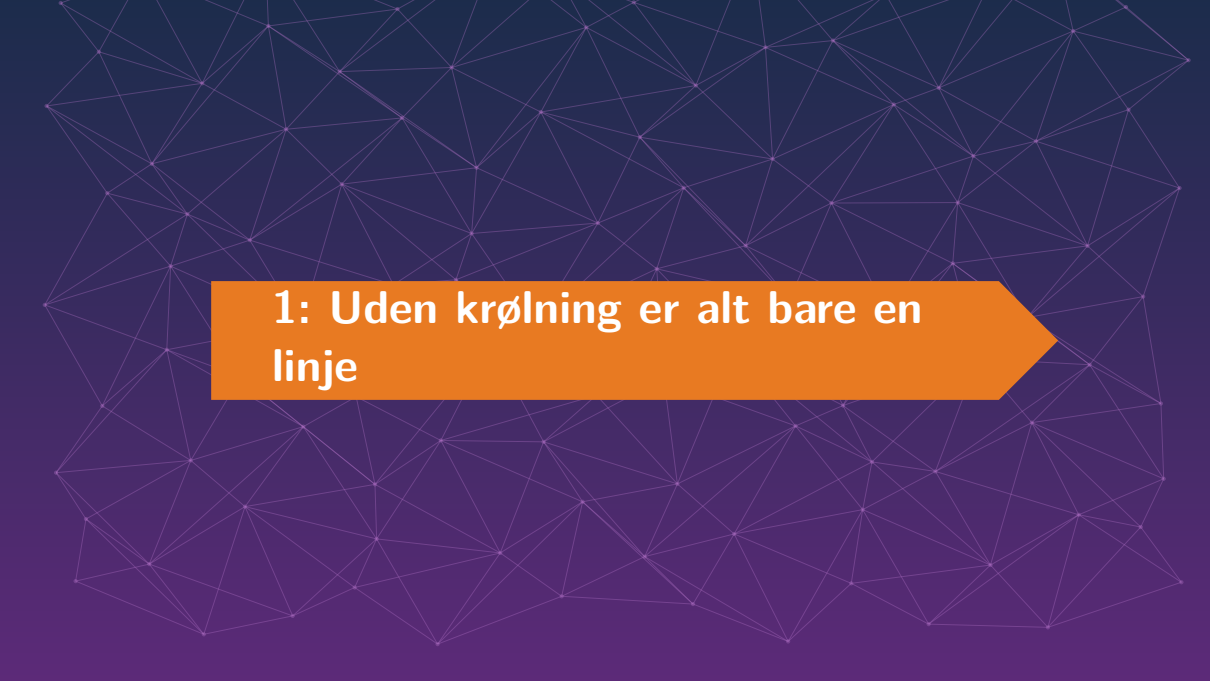
UNF | KIC 2026



Hvor kom vi fra?

- I byggede et netværk: lag af neuroner (`nn.Linear`) oven på hinanden.
- Mellem lagene sad en lille ting, vi kaldte “krølningen”: en **aktiveringsfunktion**.
- I dag: hvorfor er den så vigtig? Og hvad er forskellen på regression og klassifikation egentlig?

Spoiler: forskellen er mindre end I tror.



1: Uden krølning er alt bare en linje



Hvad sker der, hvis vi fjerner aktiveringen?

To lineære lag efter hinanden — uden krølning imellem — kan **ganges sammen** til ét:

$$W_2(W_1x + b_1) + b_2 = \underbrace{(W_2W_1)}_{W'}x + \underbrace{(W_2b_1 + b_2)}_{b'} = W'x + b'$$

- De to lag er nu **ét** lag — bare med nye vægte W' og ny bias b' .
- Med rigtige tal: $3 \cdot (2x + 1) - 2 = 6x + 1$ — stadig en ret linje.
- Ti lag uden krølning? Ganges sammen til nøjagtig én linje.

Uden noget ikke-lineært lærer selv det dybeste netværk aldrig andet end en ret streg.



Tænk på det som goombaer: en stak



en goomba-stak

- Sæt én goomba oven på en anden — en **goomba-stak**.
- Den bevæger sig stadig *præcis* som ... en goomba.
- Så det er jo bare en (lidt højere) goomba.

*To lineære lag oven på hinanden — intet imellem — =
stadig bare ét lineært lag.*

Stak ti goombaer. Stak tusind. Det er stadig ... en goomba. Ret
antiklimatisk.



Men sæt Cappy på...



goomba-stak + Cappy

- Kast **Cappy** på stakken — den nederste tager skade (bliver rød).
- Nu er den forvandlet og opfører sig helt anderledes end før.
- Det er *ikke* bare en goomba-stak længere.

Cappy = aktiveringsfunktionen. Den “krøller” outputtet, så stakken IKKE falder sammen.



...eller en sandwich imellem



- Læg en **sandwich** mellem de to goombaer.
- Nu kan de ikke stakke — der er noget imellem.
- To adskilte ting, ikke bare én goomba.

Sandwich = aktiveringen MELLEM lagene. Nu er dybden ægte — et rigtigt 2-lags netværk.

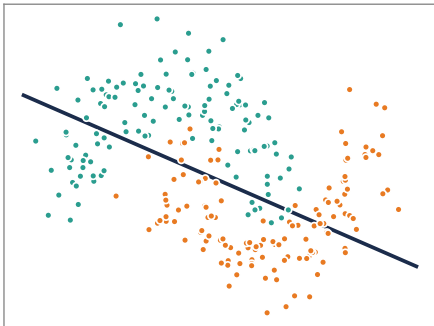
goomba · sandwich · goomba

Aktiveringsfunktionen er altså ... en sandwich. Det giver faktisk mening. På en måde.

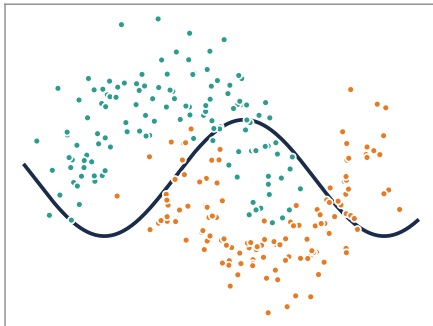


Se det med egne øjne: to måner

Uden aktivering = en lige linje



Med ReLU = en krum grænse



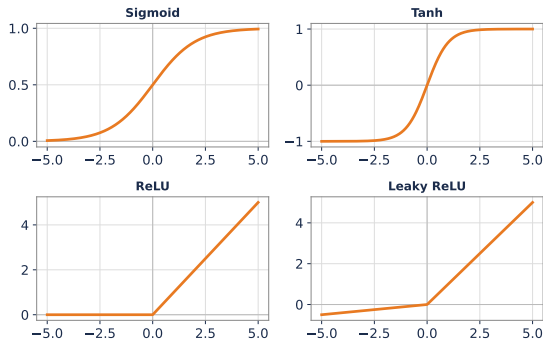
En lige streg kan aldrig dele to måner. En krum grænse kan.



2: Aktiverings-zoo



De klassiske aktiveringsfunktioner — bare linjer og kurver



- **ReLU:** negativt $\rightarrow 0$, ellers uændret. Standardvalget.
- **Sigmoid:** klemmer alt ned mellem 0 og 1.
- **Tanh:** som sigmoid, men mellem -1 og 1.
- **Leaky ReLU:** ReLU der ikke helt “dør”.

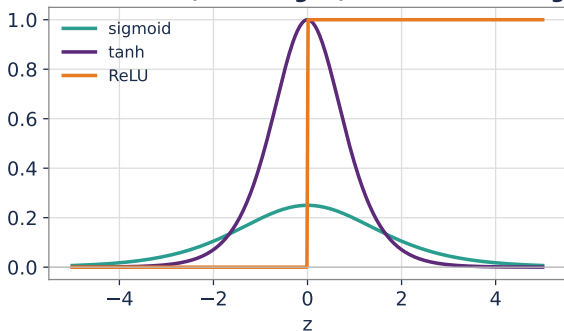
De er alle sammen bare små bøjninger — men bøjninger er nok.

Det vigtigste krav til en aktiveringsfunktion: den skal være **differentiabel** (have en hældning) — for kun så kan **gradient descent** regne baglæns igennem den (backward-passet) og opdatere vægtene.



Hvorfor er ReLU blevet standard?

Gradienten (hældningen) af hver aktivering

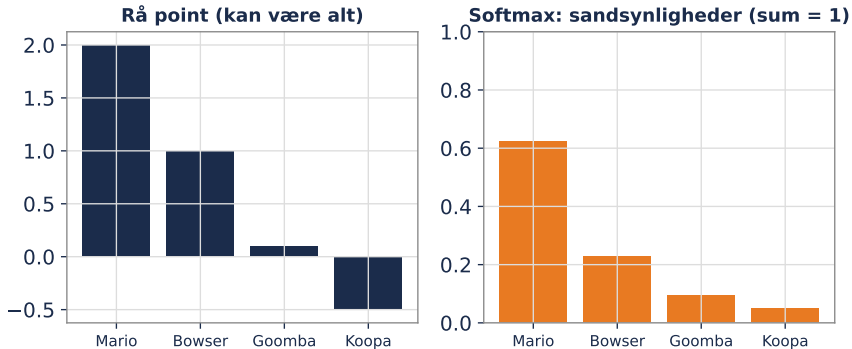


- Træning følger *hældningen* (gradienten) ned ad bakke.
- Sigmoids hældning er næsten 0 i enderne — signalet forsvinder gennem mange lag.
- ReLU har hældning 1 for positive tal — signalet slipper igennem.

Sigmoid gennem 10 lag: hviskeleg hvor beskeden bliver til ingenting.



Softmax: fra rå point til sandsynligheder

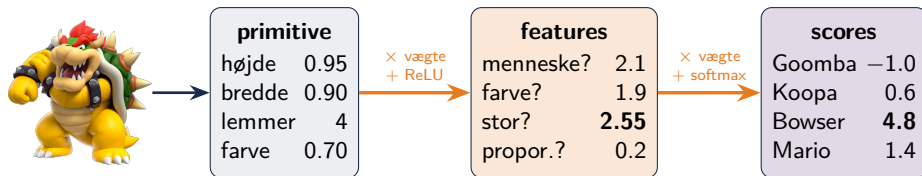


Softmax gør en bunke tal om til et pænt gæt der summer til 100 %.



Forward pass: tallene løber FREM

Fra i går: Bowers fire tal løber **frem** gennem nettet til et gæt — det kalder vi et **forward pass**.

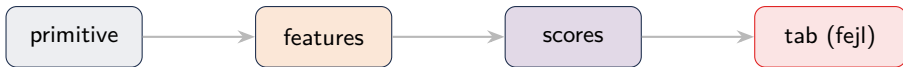


*Input → gang, læg sammen, krøl (ReLU) → gæt. Det er ét **forward pass**.*



Backward pass: fejlen løber TILBAGE

I starten er vægtene tilfældige, så gættet er forkert. Vi måler **fejlen** (tabet) og sender den **baglæns**:



gradient løber tilbage — hver vægt får et lille skub

- Hver vægt får at vide: skal jeg lidt **OP** eller lidt **NED** for at gøre fejlen mindre? (gradienten)
- Vi tager ét lille skridt — og gentager. Det er de sidste to af de fem trin: `loss.backward() + optimizer.step()`.

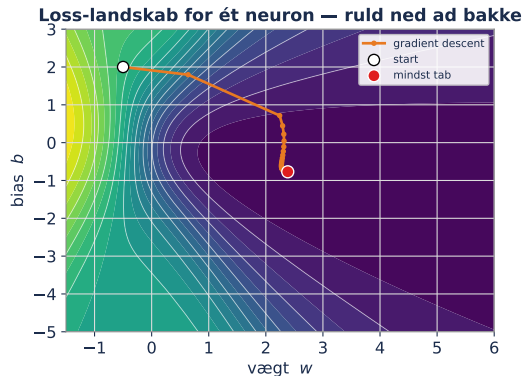
Forward for at gætte, backward for at lære. Aktiveringens hældning afgør om signalet slipper tilbage — derfor ReLU.



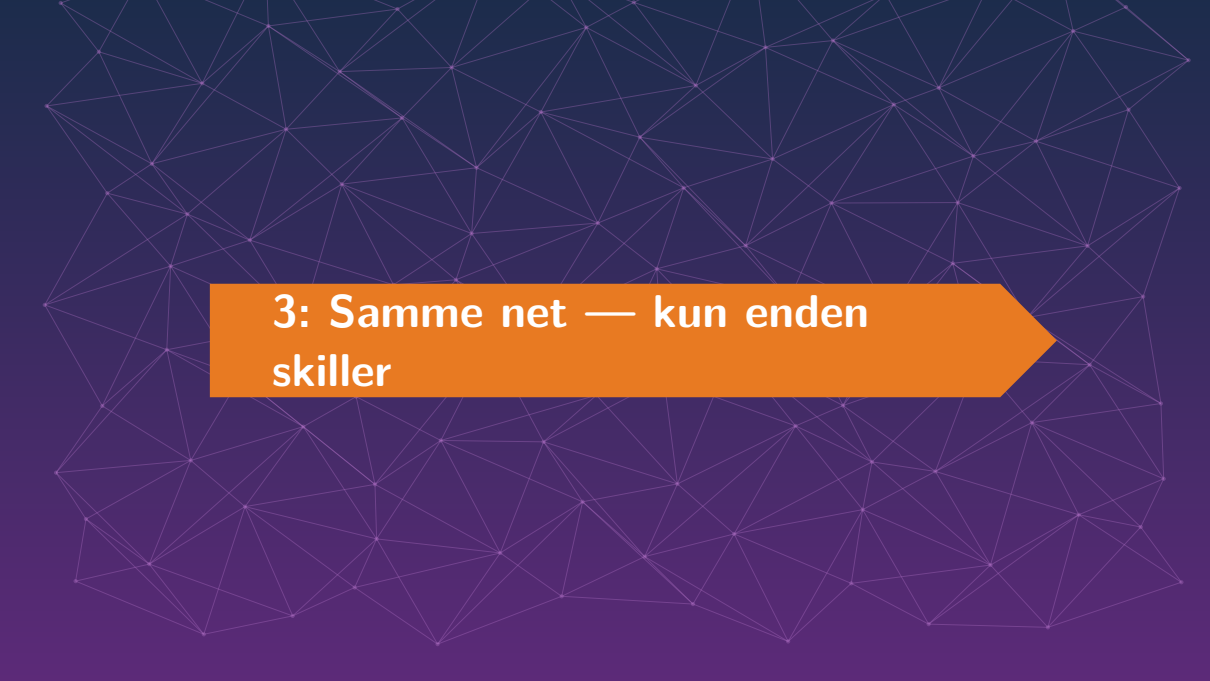
Sådan opdaterer vi: ned ad loss-landskabet

- **Loss-landskabet** viser fejlen for hver kombination af vægte.
- Gradienten peger *op* ad bakke — så vi går den modsatte vej, ét lille skridt.
- Rul ned til dalen = de bedste vægte.

Regressionens landskab var en glat skål. Et NN's er bumpet — men vi ruller stadig ned.



Se den **live-animation** i `live-coding/deck2-loss-landscape.ipynb`.

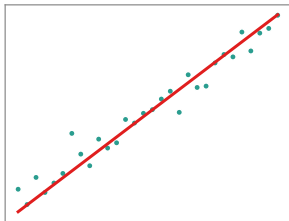


3: Samme net — kun enden skiller

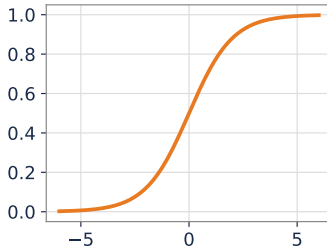


Regression og klassifikation bruger **præcis det samme netværk**. Det eneste der skifter, er den **sidste funktion** (og hvordan vi måler fejl):

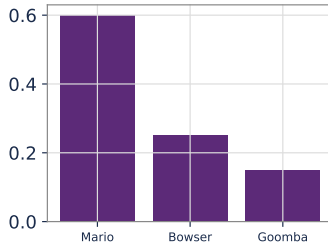
output = tal
(regression)



output = sigmoid
(2 klasser)



output = softmax
(mange klasser)

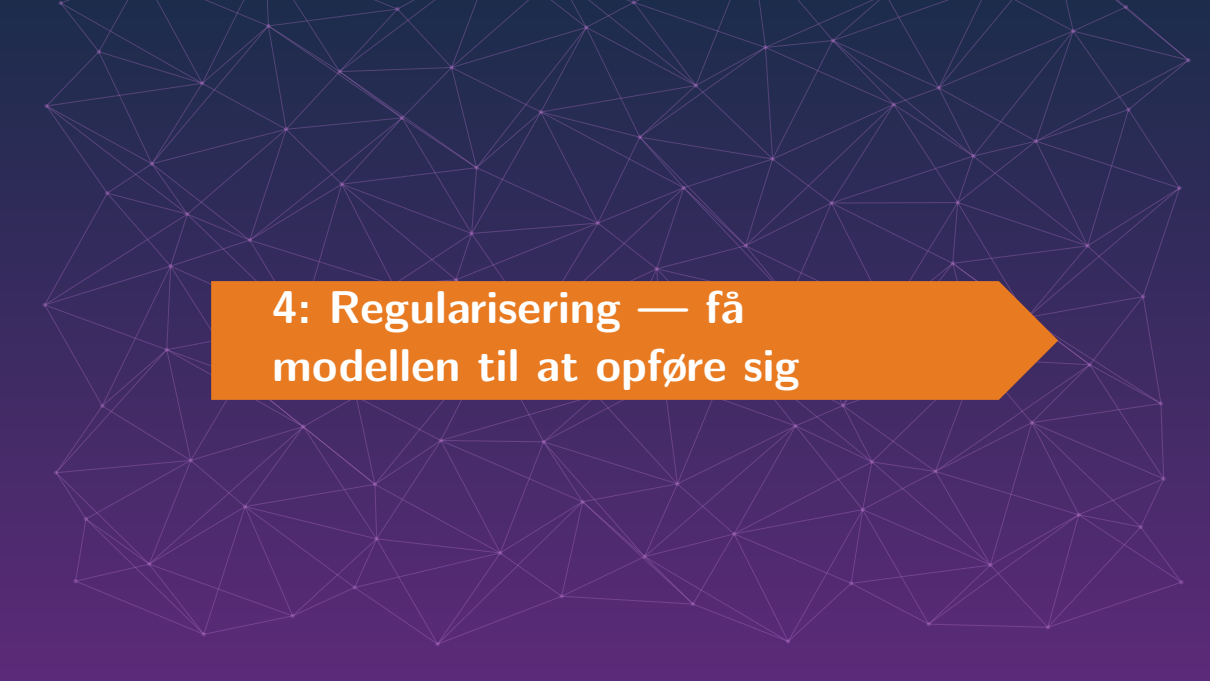




Opgrave	Output-funktion	Tabsfunktion	Eksempel
Regression	ingen (bare tallet)	MSE	is solgt, huspris
2 klasser	Sigmoid	BCELoss	legendarisk? spam?
Mange klasser	Softmax	CrossEntropyLoss	Mario/Bowser/Goomba

Matematik-varsel: CrossEntropyLoss laver selv softmax indeni — så put *ikke* softmax i modellen samtidig. Klassisk fælde!

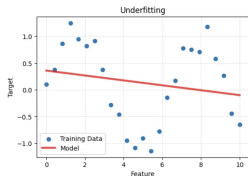
Byg netværket én gang. Skift enden. Løs en helt ny slags opgave.



4: Regularisering — få modellen til at opføre sig

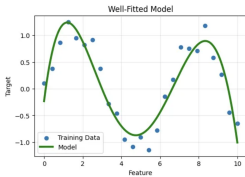


For lidt, for meget, lige tilpas



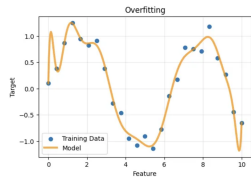
Underfitting: The model is too simple to capture the true pattern

Underfit
for doven



Well-fitted: The model closely follows the true relationship without chasing every noisy point

Lige tilpas
generaliserer



Overfitting: The model bends excessively to fit the noise instead of the underlying pattern

Overfit
udenadslære

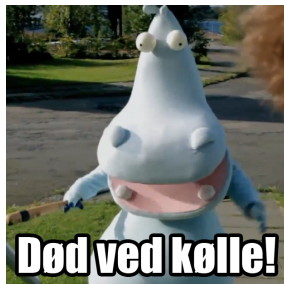
Overfit = modellen har lært træningsdata udenad, men dumper til eksamen.



Regularisering = “hold nu igen”

- Overfit sker, når modellen får lov at blive alt for indviklet.
- Regularisering er alle de tricks, der **tvinger den til at holde det simpelt**.
- Vi tager tre klassiske — ét ad gangen.

Occams ragekniv: den simpleste forklaring, der passer, vinder som regel.



Lidt disciplin, så modellen opfører sig.



Trick 1: Straf store vægte

- Store vægte laver vilde, takkede grænser, der jagter hvert enkelt datapunkt.
- Vi lægger en **bøde** oveni tabet for store vægte — så holder modellen dem små.
- Resultat: glattere grænser, der generaliserer bedre.

Det svarer til at sige: “du må godt bruge store tal — men det koster dig.”



Trick 2: Mere data

- Den sikreste kur mod udenadslære: vis den bare **flere** eksempler.
- Jo mere data, jo sværere er det at snyde ved at huske det hele udenad.
- Kan man ikke skaffe mere? Lav små variationer af det, man har (augmentering).

Svært at lære 1.000 billeder udenad. Umuligt at lære 10 millioner.



Trick 3: Dropout

- Under træning slukker vi **tilfældige** neuroner — vi sætter dem til **NUL**.
- Så kan modellen ikke læne sig op ad én enkelt neuron (fx “sandwich-features”).
- Alle skal kunne bidrage. Til test tænder vi dem alle igen.

Stol aldrig blindt på én neuron — dropout kan nulstille den når som helst.

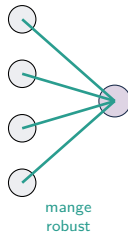
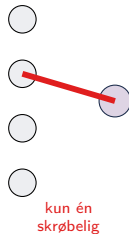




Trick 3 (fortsat): derfor generaliserer dropout

- Når neuroner slukkes tilfældigt, kan modellen **ikke over-bruge** én enkelt feature.
- Den tvinges til at sprede sig ud på **flere** features — flere veje til samme svar.
- Spredte, robuste features passer bedre på **nye** data (den generaliserer).

Ingen enkelt-feature-afhængighed → mere robust → mere generel model.





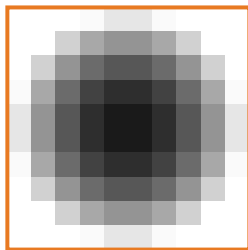
5: Dataen I kigger på



Et billede er bare tal i en tabel

- MNIST: 70.000 håndskrevne cifre, hver 28×28 pixels.
- Hver pixel er et tal fra 0 (sort) til 255 (hvid).
- Så et billede er bare 784 tal — og så virker hele værktøjsskassen fra i går.

“Billede” lyder svært. “Tabel med 784 kolonner” gør ikke.



28×28 tal $\rightarrow$ ét ciffer



MNIST — cifre

- 10 klasser (0–9)
- let nok til $\sim 95\%$ korrekt
- “Hello World” for billeder

Pokémon — tabel

- stats: HP, Attack, Defense ...
- gæt *typen* (18 klasser — svært!)
- her møder I overfitting live

At gætte en Pokémons type fra dens stats er svært — selv for netværket. Det er helt fint. Ikke alt kan læres.



Nu er det jeres tur

Nu er det jeres tur 

Åbn `03-Intro-ML/2-Activation-functions.ipynb` og gå i gang med opgaverne.
Leg med aktiveringerne, se en linje fejle på månerne, og byg 3-klasse-modeller.

Nu er det jeres tur 

Åbn `03-Intro-ML/3-MNIST-and-Pokemon.ipynb` og gå i gang med opgaverne.
Lær et netværk at læse håndskrift — og prøv den svære Pokémon-opgave.



Krøl løs — og hold øje med den sidste funktion!